Personalized Hearing Assistance MVP

A portable, software-defined prototype for hearing + tactile speech perception

> **Core idea:** use a DJI Mic Mini as the wireless microphone, an Android device as a real-time DSP engine, and a small wired speaker as the output. First measure the user's relative hearing profile; then reshape speech to emphasize or transform information into ranges the user can perceive. Later, add a tactile transducer so speech can also be felt as vibration.

1. Problem

A person with partial hearing may detect certain strong sounds - for example claps, low vocal sounds such as "OM," or helicopter-like pulsing - while ordinary speech remains difficult to perceive. Simply making speech louder is not necessarily the best solution. The prototype explores whether personalized signal processing and tactile cues can make speech easier to detect or distinguish.

2. MVP Goal

Build an Android prototype that can (1) create an experimental hearing profile, (2) accept live audio from the DJI Mic Mini, (3) process that audio with low latency, and (4) let us rapidly test which transformations improve speech perception.

MVP Screen	Purpose
Hearing Test	Generate tones by frequency and record the lowest repeatable level the user detects.
Profile	Display separate left/right relative threshold curves and save test results.
Live Voice	Capture the DJI Mic Mini over USB-C and route it through the DSP engine.
DSP Lab	Adjust EQ, compression, filtering, frequency transformation and output limiting in real time.

3. Phase A - Experimental Hearing Profile

Use wired, closed-back headphones and test each ear independently. Start with approximately **125, 250, 500, 1,000, 2,000, 4,000, 6,000 and 8,000 Hz**. At each frequency, vary the digital output level and repeat trials to estimate a reliable detection threshold.

The interface should allow responses such as **Hear clearly**, **barely hear**, **feel**, and **nothing**. Results should initially be stored in **dBFS / relative level**, not clinical dB HL, unless the complete playback chain has been properly calibrated.

Perception-window experiment

Add a second experimental test around low frequencies such as 60, 80, 100, 120, 160, 200, 250, 350 and 500 Hz. Test continuous tones, pulses and amplitude-modulated patterns. This is intended to discover ranges and temporal patterns that are especially noticeable to the user.

4. Phase B - Live DJI Voice Processing

Signal path: DJI Mic Mini transmitter -> DJI receiver -> Android via USB-C -> native DSP engine -> wired audio output -> portable powered speaker.

Avoid Bluetooth for the live processed output during the prototype because Bluetooth buffering can create perceptible latency that sounds like an echo. The target is a low-latency wired output path.

5. DSP Lab

The first DSP controls should be deliberately experimental rather than hidden behind a single 'volume' control. Useful modules include parametric EQ, high/low-pass filters, dynamic-range compression, gain, a hard output limiter, frequency lowering/transposition experiments, envelope extraction, and amplitude modulation. More creative effects such as pitch shifting or harmonic saturation can be explored, but intelligibility and safe output levels come first.

Module	Why it matters
Parametric EQ	Find and emphasize ranges that remain perceptible.
Compression	Reduce the difference between quiet and loud speech without simply maximizing volume.
Frequency transformation	Experiment with moving otherwise difficult speech information into a more usable range.
Envelope / modulation	Preserve speech rhythm and convert it into stronger audible or tactile patterns.
Limiter	Prevent the software from exceeding a defined maximum output level.

6. Phase C - Tactile Channel

After the audible prototype shows promise, add a small tactile transducer. Split the processed signal into two paths: one optimized for the speaker and one optimized for vibration. Instead of merely sending bass to the transducer, the app can encode speech amplitude, rhythm or selected spectral activity into distinct vibration patterns and test which cues the user can reliably distinguish.

> **Concept:** DJI voice -> DSP -> [speech output -> speaker] + [tactile encoding -> transducer].

7. Recommended Software Architecture

Android/Kotlin for the application UI and test workflow, with a **native C++ audio engine using Oboe/AAudio** for low-latency capture, processing and playback. Keep the DSP engine independent enough that it can later be reused on iOS or dedicated embedded hardware.

Layer	Responsibilities
Kotlin UI	Hearing test, profiles, DSP controls, presets, session results.
JNI bridge	Low-overhead communication between Android UI/state and native audio.
C++ / Oboe	Audio I/O, buffering, low-latency processing and DSP graph.

Layer	Responsibilities
DSP modules	EQ, filters, compressor, transformation, modulation, limiter.

8. Hardware for V0.1

Already owned: DJI Mic Mini. Add wired closed-back headphones for testing, a reliable USB-C/wired audio output path, and a small powered speaker with analog AUX/line input. Do not buy the tactile hardware until the initial frequency-processing experiments show useful results.

9. V0.1 Success Criteria

The MVP is successful if we can reproducibly identify frequency/pattern sensitivity and demonstrate at least one processed live-voice preset that the user perceives more clearly or consistently than unprocessed speech at a comparable safe output level. That result justifies adding tactile feedback and more advanced frequency mapping.

10. Safety and Scope

This prototype is an experimental assistive-audio research tool, not a diagnostic audiometer, hearing aid, or replacement for an audiologist. Uncalibrated consumer headphones cannot produce a clinical audiogram. Output should start conservatively, increase gradually, and always pass through a hard software limiter. A clinical audiogram can later be imported to guide the experiments more intelligently.

> **First engineering milestone:** Tone test -> relative left/right profile -> DJI USB capture -> parametric EQ + compressor + limiter -> wired speaker output.